Coffee Campus App — Actividad Formativa UML

Curso: Modelado UML y Validación Crítica con IA
Sistema: Coffee Campus App — Aplicación móvil para la cafetería universitaria

---

Tabla de Contenidos

1. Caso de Estudio
2. Fase 1 — Diagramas UML del Equipo (Mermaid)
   - Diagrama de Casos de Uso
   - Diagrama de Clases
   - Diagrama de Secuencia
   - Diagrama de Colaboración
3. Fase 2 — Co-diseño con IA
   - Prompt Inicial
   - Diagramas Generados por la IA
   - Prompt de Corrección
   - Matriz de Análisis Comparativo
4. Conclusión

---

1. Caso de Estudio

La cafetería central de la universidad requiere una aplicación móvil para reducir las filas en horas pico. El sistema debe permitir a alumnos y profesores realizar pedidos con anticipación, pagar en línea y recoger sus productos en barra.

Requerimientos Funcionales:

#	Requerimiento	Descripción
RF1	Consulta de Menú	Cualquier estudiante o profesor puede consultar la lista de productos y precios **sin** iniciar sesión.
RF2	Creación de Pedidos	El cliente autenticado selecciona productos, especifica cantidades y genera la orden.
RF3	Descuentos	Al seleccionar "Descuento Estudiantil", el sistema valida la matrícula y aplica un 10 % de rebaja al total.
RF4	Procesamiento de Pago	El sistema se conecta con una Pasarela de Pago externa. Si el pago es rechazado, se notifica al cliente y el pedido queda en estado "Pendiente".
RF5	Preparación y Entrega	Una vez confirmado el pago, el pedido pasa a la pantalla del Barista. Al finalizar, el Barista cambia el estado a "Listo para entrega" y se notifica al cliente.

---

2. Fase 1 — Diagramas UML del Equipo (Mermaid)

> *Los diagramas fuente en PlantUML se encuentran en los archivos `.puml` de la raíz del repositorio.*

2.1 Diagrama de Casos de Uso

```
flowchart LR
    Cliente(["👤 Cliente\n(Estudiante/Profesor)"])
    Barista(["👤 Barista"])
    Pasarela(["⚙️ Pasarela de Pago\n«Sistema Externo»"])
    Matricula(["⚙️ Sistema de Matrícula\n«Sistema Externo»"])
```

```
    subgraph CoffeeCampusApp["Coffee Campus App"]
        UC1(["Consultar Menú"])
        UC2(["Iniciar Sesión"])
        UC3(["Crear Pedido"])
        UC4(["Aplicar Descuento\nEstudiantil"])
        UC5(["Validar Matrícula"])
        UC6(["Procesar Pago"])
        UC7(["Preparar Pedido"])
        UC8(["Notificar Cliente"])
    end

    Cliente --> UC1
    Cliente --> UC3
    Cliente --> UC6

    UC3 -->|"«include»"| UC2
    UC4 -.->|"«extend»"| UC3
    UC4 -->|"«include»"| UC5
    UC5 --> Matricula

    UC6 --> Pasarela
    Barista --> UC7
    UC7 -->|"«include»"| UC8
```

Actores identificados:

- **Principales:** Cliente (Estudiante/Profesor), Barista
- **Secundarios / Sistemas externos:** Pasarela de Pago, Sistema de Matrícula

Relaciones destacadas:

- `«include»` : Crear Pedido incluye obligatoriamente Iniciar Sesión; Preparar Pedido incluye Notificar Cliente; Aplicar Descuento incluye Validar Matrícula.
- `«extend»` : Aplicar Descuento Estudiantil *extiende* Crear Pedido (es opcional, se activa sólo cuando el cliente lo solicita).

2.2 Diagrama de Clases

```
classDiagram
    class Usuario {
        - idUsuario : String
        - nombre : String
        - correo : String
        + iniciarSesion() boolean
        + cerrarSesion() void
    }

    class Cliente {
        - idMatricula : String
        - esEstudiante : boolean
        + consultarMenu()
        + solicitarDescuento() boolean
    }

    class Barista {
        - idTurno : String
```

```
        + verPedidosPendientes()
        + actualizarEstadoPedido()
    }

    class Producto {
        - idProducto : String
        - nombre : String
        - precio : double
        - disponible : boolean
        + getPrecio() double
    }

    class Pedido {
        - idPedido : String
        - fecha : Date
        - estado : String
        - total : double
        + agregarDetalle()
        + aplicarDescuento() void
        + calcularTotal() double
    }

    class DetallePedido {
        - cantidad : int
        - subtotal : double
        + calcularSubtotal() double
    }

    class Pago {
        - idPago : String
        - monto : double
        - estado : String
        + procesarTransaccion()
    }

    Usuario <|-- Cliente
    Usuario <|-- Barista

    Cliente "1" --> "0..*" Pedido : realiza
    Pedido "1" *-- "1..*" DetallePedido : contiene
    DetallePedido "0..*" --> "1" Producto : refiere a
    Pedido "1" --> "1" Pago : registra
```

Tipos de relaciones utilizadas:

- **Herencia (`<|--`):** `Cliente` y `Barista` heredan de `Usuario` .
- **Composición (`*--`):** `Pedido` está compuesto de `DetallePedido` (el detalle no existe sin el pedido).
- **Asociación (`-->`):** `Cliente` realiza `Pedido` ; `DetallePedido` refiere a `Producto` ; `Pedido` registra un `Pago` .

2.3 Diagrama de Secuencia

Escenario: Realizar Pedido y Procesar Pago (flujo exitoso)

```
sequenceDiagram
    actor Cliente
    participant UI as interfaz : AppUI
```

```
    participant CP as ctrlPedido : ControladorPedido
    participant CPago as ctrlPago : ControladorPago
    participant Pedido as p : Pedido
    participant Pasarela as Pasarela de Pago

    Cliente ->> UI: seleccionarProductos()
    activate UI
    UI ->> CP: iniciarPedido(listaProductos)
    activate CP
    CP ->> Pedido: new Pedido(listaProductos)
    activate Pedido
    Pedido -->> CP: pedidoCreado
    deactivate Pedido
    CP -->> UI: mostrarTotal(total)
    deactivate CP

    Cliente ->> UI: confirmarYpagar()
    UI ->> CPago: procesarPago(idPedido, datosTarjeta)
    activate CPago
    CPago ->> Pasarela: autorizarTransaccion(monto, datosTarjeta)
    activate Pasarela
    Pasarela -->> CPago: transaccionAprobada
    deactivate Pasarela

    CPago ->> Pedido: actualizarEstado("Pagado")
    activate Pedido
    Pedido -->> CPago: estadoActualizado
    deactivate Pedido

    CPago -->> UI: pagoExitoso()
    deactivate CPago
    UI -->> Cliente: mostrarRecibo(idPedido)
    deactivate UI
```

2.4 Diagrama de Colaboración

Escenario: Realizar Pedido y Procesar Pago

> *Mermaid no posee soporte nativo para diagramas de comunicación/colaboración UML. Se representa mediante un diagrama de flujo con los mensajes numerados cronológicamente sobre los enlaces, respetando la semántica del diagrama de colaboración UML.*

```
flowchart LR
    Cliente(["👤 Cliente"])
    UI["interfaz : AppUI"]
    CP["ctrlPedido :\nControladorPedido"]
    CPago["ctrlPago :\nControladorPago"]
    Pedido["p : Pedido"]
    Pasarela["pasarela :\nPasarela de Pago"]

    Cliente -- "1: seleccionarProductos()\n5: confirmarYpagar()" --> UI
    UI -- "2: iniciarPedido(lista)" --> CP
    CP -- "3: crear(lista)" --> Pedido
    CP -- "4: mostrarTotal(total)" --> UI
    UI -- "6: procesarPago(id)" --> CPago
    CPago -- "7: autorizarTransaccion()" --> Pasarela
    Pasarela -- "8: transaccionAprobada()" --> CPago
```

```
    CPago -- "9: actualizarEstado('Pagado')" --> Pedido
    CPago -- "10: pagoExitoso()" --> UI
    UI -- "11: mostrarRecibo(id)" --> Cliente
```

---

3. Fase 2 — Co-diseño con IA

3.1 Prompt Inicial

Se utilizó **Gemini** con el siguiente prompt:

> ***Prompt inicial enviado a Gemini:***
>
> *Eres un experto en modelado UML. A partir de los siguientes requerimientos, genera los 4 diagramas UML en código Mermaid, listos para renderizar en Mermaid Live:*
>
> ***Caso de estudio: Coffee Campus App***
> *La cafetería de una universidad necesita una app móvil. Los requerimientos son:*
>
> 1. *Cualquier usuario puede consultar el menú sin iniciar sesión.*
> 2. *El cliente autenticado puede crear pedidos seleccionando productos y cantidades.*
> 3. *Si el cliente elige "Descuento Estudiantil", el sistema valida su matrícula con un sistema externo y aplica un 10% de descuento. Esta opción es opcional y extiende el flujo de creación de pedido.*
> 4. *El pago se procesa a través de una Pasarela de Pago externa. Si es rechazado, el pedido queda en "Pendiente".*
> 5. *Al confirmar el pago, el Barista ve el pedido y cuando lo prepara cambia el estado a "Listo para entrega", notificando al cliente.*
>
> *Genera:*
>
> 1. ***Diagrama de Casos de Uso** con actores principales (Cliente, Barista) y sistemas externos (Pasarela de Pago, Sistema de Matrícula). Usa relaciones `«include»` y `«extend»` correctamente.*
> 2. ***Diagrama de Clases** con todas las entidades (Usuario, Cliente, Barista, Producto, Pedido, DetallePedido, Pago), sus atributos privados (-) y métodos públicos (+), y las relaciones con multiplicidades (herencia, composición, asociación).*
> 3. ***Diagrama de Secuencia** modelando el escenario exitoso "Realizar Pedido y Procesar Pago", con mensajes de retorno (líneas punteadas), activaciones y participación de la Pasarela de Pago.*
> 4. ***Diagrama de Colaboración** con los mismos objetos del diagrama de secuencia, numerando explícitamente el orden cronológico de cada mensaje sobre los enlaces.*

3.2 Diagramas Generados por la IA

A continuación se presentan los diagramas tal como fueron generados por Gemini en su respuesta inicial.

Casos de Uso (generado por IA)

```
flowchart LR
    Cliente(["Cliente (Estudiante/Profesor)"])
    Barista(["Barista"])
    PasarelaPago(["Pasarela de Pago (Sistema Externo)"])
    SistemaMatricula(["Sistema de Matrícula (Sistema Externo)"])

    subgraph CoffeeCampusApp["Coffee Campus App"]
        UC1["Consultar Menú"]
        UC2["Iniciar Sesión"]
        UC3["Crear Pedido"]
        UC4["Aplicar Descuento Estudiantil"]
        UC5["Validar Matrícula"]
        UC6["Procesar Pago"]
```

```
        UC7["Preparar Pedido"]
        UC8["Notificar Cliente"]
    end

    Cliente --> UC1
    Cliente --> UC2
    Cliente --> UC3
    Cliente --> UC6

    UC3 -- "«include»" --> UC2
    UC3 -- "«include»" --> UC4
    UC4 -- "«include»" --> UC5
    UC5 --> SistemaMatricula
    UC6 --> PasarelaPago
    Barista --> UC7
    UC7 -- "«include»" --> UC8
```

Diagrama de Clases (generado por IA)

```
classDiagram
    class Usuario {
        -idUsuario: int
        -nombre: String
        -email: String
        -contrasena: String
        +iniciarSesion(email, contrasena): boolean
        +cerrarSesion(): void
        +getIdUsuario(): int
        +getNombre(): String
    }
    class Cliente {
        -idMatricula: String
        -esEstudiante: boolean
        -historialPedidos: List~Pedido~
        +consultarMenu(): List~Producto~
        +crearPedido(productos: List~Producto~): Pedido
        +solicitarDescuento(): boolean
        +verHistorialPedidos(): List~Pedido~
    }
    class Barista {
        -idEmpleado: String
        -turno: String
        +verPedidosPendientes(): List~Pedido~
        +actualizarEstadoPedido(idPedido: String, nuevoEstado: String): void
        +marcarListoParaEntrega(idPedido: String): void
    }
    class Producto {
        -idProducto: String
        -nombre: String
        -descripcion: String
        -precio: double
        -disponible: boolean
        -categoria: String
        +getPrecio(): double
        +isDisponible(): boolean
```

```
        +getNombre(): String
    }
    class Pedido {
        -idPedido: String
        -fechaHora: DateTime
        -estado: String
        -totalSinDescuento: double
        -totalConDescuento: double
        -descuentoAplicado: boolean
        +agregarDetalle(producto: Producto, cantidad: int): void
        +aplicarDescuento(porcentaje: double): void
        +calcularTotal(): double
        +getEstado(): String
        +setEstado(nuevoEstado: String): void
    }
    class DetallePedido {
        -idDetalle: String
        -cantidad: int
        -precioUnitario: double
        -subtotal: double
        +calcularSubtotal(): double
        +getCantidad(): int
        +getPrecioUnitario(): double
    }
    class Pago {
        -idPago: String
        -monto: double
        -metodoPago: String
        -estado: String
        -fechaTransaccion: DateTime
        +procesarTransaccion(): boolean
        +getEstado(): String
        +getMonto(): double
    }

    Usuario <|-- Cliente
    Usuario <|-- Barista
    Cliente "1" --> "0..*" Pedido : realiza
    Pedido "1" *-- "1..*" DetallePedido : contiene
    DetallePedido "0..*" --> "1" Producto : referencia
    Pedido "1" --> "1" Pago : genera
```

Diagrama de Secuencia (generado por IA)

```
sequenceDiagram
    actor Cliente
    participant AppUI as AppUI
    participant ControladorPedido as ControladorPedido
    participant Pedido as Pedido
    participant ControladorPago as ControladorPago
    participant PasarelaPago as Pasarela de Pago

    Cliente->>AppUI: seleccionarProductos(listaProductos)
    AppUI->>ControladorPedido: iniciarPedido(listaProductos)
    ControladorPedido->>Pedido: new Pedido(listaProductos)
```

```
    Pedido-->>ControladorPedido: pedidoCreado (idPedido, total)
    ControladorPedido-->>AppUI: mostrarResumenPedido(idPedido, total)
    AppUI-->>Cliente: Muestra resumen y total del pedido

    Cliente->>AppUI: confirmarPedidoYPagar(idPedido, datosPago)
    AppUI->>ControladorPago: procesarPago(idPedido, datosPago)
    ControladorPago->>PasarelaPago: autorizarTransaccion(monto, datosPago)
    PasarelaPago-->>ControladorPago: respuestaAutorizacion (aprobado/rechazado)

    alt Pago Aprobado
        ControladorPago->>Pedido: actualizarEstado("Pagado")
        Pedido-->>ControladorPago: estadoActualizado
        ControladorPago-->>AppUI: pagoExitoso(idPedido, recibo)
        AppUI-->>Cliente: mostrarRecibo(recibo)
    else Pago Rechazado
        ControladorPago->>Pedido: actualizarEstado("Pendiente")
        Pedido-->>ControladorPago: estadoActualizado
        ControladorPago-->>AppUI: pagoRechazado(motivo)
        AppUI-->>Cliente: notificarRechazo(motivo)
    end
```

Diagrama de Colaboración (generado por IA)

```
flowchart TD
    Cliente(["Cliente"])
    AppUI["AppUI"]
    ControladorPedido["ControladorPedido"]
    Pedido["Pedido"]
    ControladorPago["ControladorPago"]
    PasarelaPago["Pasarela de Pago"]

    Cliente -->|"1: seleccionarProductos(listaProductos)"| AppUI
    AppUI -->|"2: iniciarPedido(listaProductos)"| ControladorPedido
    ControladorPedido -->|"3: new Pedido(listaProductos)"| Pedido
    Pedido -->|"4: pedidoCreado(idPedido, total)"| ControladorPedido
    ControladorPedido -->|"5: mostrarResumenPedido(idPedido, total)"| AppUI
    AppUI -->|"6: Muestra resumen"| Cliente
    Cliente -->|"7: confirmarPedidoYPagar(idPedido, datosPago)"| AppUI
    AppUI -->|"8: procesarPago(idPedido, datosPago)"| ControladorPago
    ControladorPago -->|"9: autorizarTransaccion(monto, datosPago)"| PasarelaPago
    PasarelaPago -->|"10: respuestaAutorizacion(aprobado)"| ControladorPago
    ControladorPago -->|"11: actualizarEstado('Pagado')"| Pedido
    Pedido -->|"12: estadoActualizado"| ControladorPago
    ControladorPago -->|"13: pagoExitoso(recibo)"| AppUI
    AppUI -->|"14: mostrarRecibo(recibo)"| Cliente
```

3.3 Prompt de Corrección

Tras analizar los diagramas generados por la IA, se identificaron los errores descritos en la matriz (sección 3.4) y se envió el siguiente prompt de corrección:

> ***Prompt de corrección enviado a Gemini:***
>
> *Los diagramas que generaste tienen varios errores UML semánticos. Por favor corrígelos:*

> ***Error 1 — Casos de Uso:***
> *La relación entre "Crear Pedido" y "Aplicar Descuento Estudiantil" debe ser `«extend»`, NO `«include»`. El descuento es **opcional** y sólo se activa si el cliente lo solicita. Un `«include»` implica que siempre se ejecuta obligatoriamente. Corrige esto y elimina la asociación directa `Cliente --> UC2` porque el cliente no navega directamente a "Iniciar Sesión"; lo hace a través de "Crear Pedido" que lo incluye.*
>
> ***Error 2 — Diagrama de Clases:***
> *El diagrama está bien estructuralmente, pero añadiste atributos y métodos que no forman parte del modelo acordado (e.g., `historialPedidos`, `contrasena`, `descripcion`, `categoria`, `getters` innecesarios). Mantenlos al mínimo requerido por los requerimientos. Los atributos privados se marcan con `-` y los métodos públicos con `+`, como ya tenías.*
>
> ***Error 3 — Diagrama de Secuencia:***
> *El diagrama es aceptable pero debes asegurarte de que el nombre de los participantes use la notación UML de instancia: `nombreInstancia : NombreClase` (e.g., `interfaz : AppUI`, `ctrlPedido : ControladorPedido`). Esto los distingue de clases.*
>
> ***Error 4 — Diagrama de Colaboración:***
> *El diagrama usa orientación `TD` (top-down) en lugar de `LR`, lo que dificulta leer los enlaces. Más importante: los mensajes de retorno (respuestas) también deben estar numerados en el flujo cronológico. Los objetos intermedios no deben retornar mensajes de vuelta como flechas separadas cuando no aportan al flujo principal del escenario de éxito.*

3.4 Matriz de Análisis Comparativo

Aspecto Evaluado	Diagrama Original (Equipo)	Diagrama Generado (IA)	Hallazgo / Error de la IA
Casos de Uso	`«extend»` correcto: "Aplicar Descuento" extiende "Crear Pedido" de forma opcional. Cliente **no** apunta directamente a "Iniciar Sesión".	La IA usó `«include»` entre "Crear Pedido" y "Aplicar Descuento", haciendo el descuento **obligatorio**. Además añadió `Cliente --> IniciarSesión` como asociación directa.	**Error semántico grave:** confundió `«extend»` con `«include»`. No entendió que `«extend»` modela comportamiento opcional/condicional, mientras que `«include»` es obligatorio.
Clases	Atributos y métodos mínimos y precisos según los requerimientos. Visibilidad explícita (+ / -). Relaciones correctas con multiplicidades.	Incluyó atributos adicionales no requeridos (`contrasena`, `historialPedidos`, `categoria`, `descripcion`) y numerosos getters/setters que inflan el modelo.	**Error de sobre-especificación:** la IA "completó" el modelo más allá de los requerimientos dados, introduciendo ruido. Omitió el atributo `fecha: Date` del Pedido reemplazándolo por `fechaHora: DateTime`.
Secuencia	Participantes nombrados con notación de instancia UML (`interfaz : AppUI`). Mensajes de retorno (líneas punteadas) correctamente representados. Pasarela de Pago incluida.	Los participantes se nombraron sin notación de instancia (sólo el nombre de clase). El flujo de pago rechazado fue añadido sin que se pidiera. La Pasarela sí fue incluida.	**Error de notación:** omitió la nomenclatura de instancia UML. **Error de alcance:** modeló el flujo alternativo (pago rechazado) mezclado con el flujo exitoso solicitado, complicando innecesariamente el diagrama.
Colaboración	Flujo `LR`, mensajes numerados sobre los enlaces. Sólo mensajes relevantes al escenario exitoso.	Usó orientación `TD`. Numeró los mensajes de retorno como pasos separados (4, 6, 12, 14), desvirtuando el flujo del escenario principal.	**Error de estructura:** al numerar los mensajes de retorno como pasos independientes, el diagrama duplica acciones y confunde el conteo cronológico. En el diagrama de colaboración, los retornos se representan sobre el mismo enlace o se omiten si son implícitos.

---

4. Conclusión

El ejercicio de co-diseño con IA evidenció que las herramientas de IA generativa como Gemini demuestran una comprensión sólida de la **sintaxis** de los lenguajes de modelado —produciendo código Mermaid válido y estructuralmente coherente— pero presentan limitaciones significativas en la **semántica UML**. El error más revelador fue la confusión sistemática entre `«include»` y `«extend»`: la IA trató como obligatorio un comportamiento que, por definición del requerimiento, es condicional y opcional. De igual forma, la tendencia a "completar" el modelo de clases con atributos y métodos no solicitados refleja que la IA optimiza para la exhaustividad sintáctica en lugar de la precisión conceptual del dominio. Esto confirma que la IA es una herramienta útil de apoyo para agilizar la notación, pero no puede reemplazar el juicio del modelador humano, quien entiende el **por qué** detrás de cada decisión de diseño, no sólo el **cómo** escribirla.

Repositorio generado como entregable de la Actividad Formativa — Modelado UML y Validación Crítica con IA.